NIST Special Publication 800-218

Secure Software Development Framework (SSDF) Version 1.1:

Recommendations for Mitigating the Risk of Software Vulnerabilities

Murugiah Souppaya
Karen Scarfone
Donna Dodson

This publication is available free of charge from:
https://doi.org/10.6028/NIST.SP.800-218

National Institute of Standards and Technology
U.S. Department of Commerce

NIST Special Publication 800-218

Secure Software Development Framework (SSDF) Version 1.1:

Recommendations for Mitigating the Risk of Software Vulnerabilities

Murugiah Souppaya
Computer Security Division
Information Technology Laboratory

Karen Scarfone
Scarfone Cybersecurity
Clifton, VA

Donna Dodson*
* *Former NIST employee; all work for this publication was done while at NIST.*

This publication is available free of charge from:
https://doi.org/10.6028/NIST.SP.800-218

February 2022



U.S. Department of Commerce
Gina M. Raimondo, Secretary

National Institute of Standards and Technology
James K. Olthoff, Performing the Non-Exclusive Functions and Duties of the Under Secretary of Commerce for Standards and Technology & Director, National Institute of Standards and Technology

Authority

This publication has been developed by NIST in accordance with its statutory responsibilities under the Federal Information Security Modernization Act (FISMA) of 2014, 44 U.S.C. § 3551 *et seq.*, Public Law (P.L.) 113-283. NIST is responsible for developing information security standards and guidelines, including minimum requirements for federal information systems, but such standards and guidelines shall not apply to national security systems without the express approval of appropriate federal officials exercising policy authority over such systems. This guideline is consistent with the requirements of the Office of Management and Budget (OMB) Circular A-130.

Nothing in this publication should be taken to contradict the standards and guidelines made mandatory and binding on federal agencies by the Secretary of Commerce under statutory authority. Nor should these guidelines be interpreted as altering or superseding the existing authorities of the Secretary of Commerce, Director of the OMB, or any other federal official. This publication may be used by nongovernmental organizations on a voluntary basis and is not subject to copyright in the United States. Attribution would, however, be appreciated by NIST.

National Institute of Standards and Technology Special Publication 800-218
Natl. Inst. Stand. Technol. Spec. Publ. 800-218, 36 pages (February 2022)
CODEN: NSPUE2

This publication is available free of charge from:
https://doi.org/10.6028/NIST.SP.800-218

Certain commercial entities, equipment, or materials may be identified in this document in order to describe an experimental procedure or concept adequately. Such identification is not intended to imply recommendation or endorsement by NIST, nor is it intended to imply that the entities, materials, or equipment are necessarily the best available for the purpose.

There may be references in this publication to other publications currently under development by NIST in accordance with its assigned statutory responsibilities. The information in this publication, including concepts and methodologies, may be used by federal agencies even before the completion of such companion publications. Thus, until each publication is completed, current requirements, guidelines, and procedures, where they exist, remain operative. For planning and transition purposes, federal agencies may wish to closely follow the development of these new publications by NIST.

Organizations are encouraged to review all draft publications during public comment periods and provide feedback to NIST. Many NIST cybersecurity publications, other than the ones noted above, are available at https://csrc.nist.gov/publications.

Submit comments on this publication to: ssdf@nist.gov

National Institute of Standards and Technology
Attn: Computer Security Division, Information Technology Laboratory
100 Bureau Drive (Mail Stop 8930) Gaithersburg, MD 20899-8930

All comments are subject to release under the Freedom of Information Act (FOIA).

Reports on Computer Systems Technology

The Information Technology Laboratory (ITL) at the National Institute of Standards and Technology (NIST) promotes the U.S. economy and public welfare by providing technical leadership for the Nation's measurement and standards infrastructure. ITL develops tests, test methods, reference data, proof of concept implementations, and technical analyses to advance the development and productive use of information technology. ITL's responsibilities include the development of management, administrative, technical, and physical standards and guidelines for the cost-effective security and privacy of other than national security-related information in federal information systems. The Special Publication 800-series reports on ITL's research, guidelines, and outreach efforts in information system security, and its collaborative activities with industry, government, and academic organizations.

This publication is available free of charge from: https://doi.org/10.6028/NIST.SP.800-218

Abstract

Few software development life cycle (SDLC) models explicitly address software security in detail, so secure software development practices usually need to be added to each SDLC model to ensure that the software being developed is well-secured. This document recommends the Secure Software Development Framework (SSDF) – a core set of high-level secure software development practices that can be integrated into each SDLC implementation. Following such practices should help software producers reduce the number of vulnerabilities in released software, reduce the potential impact of the exploitation of undetected or unaddressed vulnerabilities, and address the root causes of vulnerabilities to prevent future recurrences. Because the framework provides a common vocabulary for secure software development, software acquirers can also use it to foster communications with suppliers in acquisition processes and other management activities.

Keywords

secure software development; Secure Software Development Framework (SSDF); secure software development practices; software acquisition; software development; software development life cycle (SDLC); software security.

Trademark Information

All registered trademarks or trademarks belong to their respective organizations.

Acknowledgments

The authors thank all of the organizations and individuals who provided input for this update to the SSDF. In response to Section 4 of Executive Order (EO) 14028 on "Improving the Nation's Cybersecurity," NIST held a June 2021 workshop and received over 150 position papers, many of which suggested secure software development practices, tasks, examples of implementations, and references for consideration for this SSDF update. The authors appreciate all of those suggestions, as well as the input from those who spoke at or attended the workshop and shared their thoughts during or after the workshop.

Additionally, the authors appreciate the public comments submitted by dozens of organizations and individuals and wish to acknowledge the particularly helpful feedback from Amazon Web Services, Apiiro, Blackberry, BSA | The Software Alliance, the Enterprise Cloud Coalition, the General Services Administration (GSA), Google, IBM, Medical Imaging & Technology Alliance (MITA), Microsoft, Oracle, the Software Assurance Forum for Excellence in Code (SAFECode), Synopsis, the U.S. Navy, Xoomworks, and Robert Grupe. Representatives of Siemens Energy and Synopsis contributed mappings to new references.

The authors thank all of their NIST colleagues for their support throughout the SSDF update, especially Curt Barker, Paul Black, Jon Boyens, Jim Foti, Barbara Guttman, Mat Heyman, Nicole Keller, Matt Scholl, Adam Sedgewick, Kevin Stine, and Isabel Van Wyk.

The authors also wish to thank all of the individuals and organizations who provided comments on drafts of the original version of the SSDF, including the Administrative Offices of the U.S. Courts, The Aerospace Corporation, BSA | The Software Alliance, Capitis Solutions, the Consortium for Information & Software Quality (CISQ), HackerOne, Honeycomb Secure Systems, iNovex, Ishpi Information Technologies, the Information Security and Privacy Advisory Board (ISPAB), Juniper Networks, Microsoft, MITA, Naval Sea Systems Command (NAVSEA), NIST, Northrop Grumman, the Office of the Undersecretary of Defense for Research and Engineering, Red Hat, SAFECode, and the Software Engineering Institute (SEI).

Audience

There are two primary audiences for this document. The first is software producers (e.g., commercial-off-the-shelf [COTS] product vendors, government-off-the-shelf [GOTS] software developers, custom software developers, internal development teams) regardless of size, sector, or level of maturity. The second is software acquirers – both federal agencies and other organizations. Readers of this document are not expected to be experts in secure software development in order to understand it, but such expertise is required to implement its recommended practices.

Personnel within the following Workforce Categories and Specialty Areas from the National Initiative for Cybersecurity Education (NICE) Cybersecurity Workforce Framework [SP800181] are most likely to find this publication of interest:

This publication is available free of charge from: https://doi.org/10.6028/NIST.SP.800-218

- Securely Provision (SP): Risk Management (RSK), Software Development (DEV), Systems Requirements Planning (SRP), Test and Evaluation (TST), Systems Development (SYS)
- Operate and Maintain (OM): Systems Analysis (ANA)
- Oversee and Govern (OV): Training, Education, and Awareness (TEA); Cybersecurity Management (MGT); Executive Cyber Leadership (EXL); Program/Project Management (PMA) and Acquisition
- Protect and Defend (PR): Incident Response (CIR), Vulnerability Assessment and Management (VAM)
- Analyze (AN): Threat Analysis (TWA), Exploitation Analysis (EXP)

Note to Readers

We encourage you to provide feedback on the SSDF at any time, especially as you implement the SSDF within your own organization and software development efforts. Having inputs from a variety of software producers will be particularly helpful to us in refining and revising the SSDF. The publication will be updated periodically to reflect your inputs and feedback.

If you are from a standards-developing organization or another organization that has produced a set of secure practices and you would like to map your secure software development standard or guidance to the SSDF, please contact us at ssdf@nist.gov. We would like to introduce you to the National Online Informative References Program (OLIR) so that you can submit your mapping there to augment the existing set of informative references.

This publication is available free of charge from: https://doi.org/10.6028/NIST.SP.800-218

Patent Disclosure Notice

NOTICE: The Information Technology Laboratory (ITL) has requested that holders of patent claims whose use may be required for compliance with the guidance or requirements of this publication disclose such patent claims to ITL. However, holders of patents are not obligated to respond to ITL calls for patents and ITL has not undertaken a patent search in order to identify which, if any, patents may apply to this publication.

As of the date of publication and following call(s) for the identification of patent claims whose use may be required for compliance with the guidance or requirements of this publication, no such patent claims have been identified to ITL.

No representation is made or implied by ITL that licenses are not required to avoid patent infringement in the use of this publication.

This publication is available free of charge from: https://doi.org/10.6028/NIST.SP.800-218

Executive Summary

This document describes a set of fundamental, sound practices for secure software development called the Secure Software Development Framework (SSDF). Organizations should integrate the SSDF throughout their existing software development practices, express their secure software development requirements to third-party suppliers using SSDF conventions, and acquire software that meets the practices described in the SSDF. Using the SSDF helps organizations to meet the following secure software development recommendations:

- Organizations should ensure that their people, processes, and technology are prepared to perform secure software development.
- Organizations should protect all components of their software from tampering and unauthorized access.
- Organizations should produce well-secured software with minimal security vulnerabilities in its releases.
- Organizations should identify residual vulnerabilities in their software releases and respond appropriately to address those vulnerabilities and prevent similar ones from occurring in the future.

The SSDF does not prescribe how to implement each practice. The focus is on the outcomes of the practices rather than on the tools, techniques, and mechanisms to do so. This means that the SSDF can be used by organizations in any sector or community, regardless of size or cybersecurity sophistication. It can also be used for any type of software development, regardless of technology, platform, programming language, or operating environment.

The SSDF defines only a high-level subset of what organizations may need to do, so organizations should consult the references and other resources for additional information on implementing the practices. Not all practices are applicable to all use cases; organizations should adopt a risk-based approach to determine what practices are relevant, appropriate, and effective to mitigate the threats to their software development practices.

Organizations can communicate how they are addressing the clauses from Section 4 of the President's Executive Order (EO) on "Improving the Nation's Cybersecurity (14028)" by referencing the SSDF practices and tasks described in Appendix A.

This publication is available free of charge from: https://doi.org/10.6028/NIST.SP.800-218

Table of Contents

Executive Summary ..... **vi**
1 Introduction **1**
2 The Secure Software Development Framework **4**
References **20**
Appendix A— The SSDF and Executive Order 14028 ..... **24**
Appendix B— Acronyms **25**
Appendix C— Change Log **27**

List of Tables

Table 1: The Secure Software Development Framework (SSDF) Version 1.1 5
Table 2: SSDF Practices Corresponding to EO 14028 Clauses 24

This publication is available free of charge from: https://doi.org/10.6028/NIST.SP.800-218

1 Introduction

A *software development life cycle (SDLC)*[1] is a formal or informal methodology for designing, creating, and maintaining software (including code built into hardware). There are many models for SDLCs, including waterfall, spiral, agile, and – in particular – agile combined with software development and IT operations (DevOps) practices. Few SDLC models explicitly address software security in detail, so secure software development practices usually need to be added to and integrated into each SDLC model. Regardless of which SDLC model is used, secure software development practices should be integrated throughout it for three reasons: to reduce the number of vulnerabilities in released software, to reduce the potential impact of the exploitation of undetected or unaddressed vulnerabilities, and to address the root causes of vulnerabilities to prevent recurrences. Vulnerabilities include not just bugs caused by coding flaws, but also weaknesses caused by security configuration settings, incorrect trust assumptions, and outdated risk analysis. [IR7864]

Most aspects of security can be addressed multiple times within an SDLC, but in general, the earlier in the SDLC that security is addressed, the less effort and cost is ultimately required to achieve the same level of security. This principle, known as *shifting left*, is critically important regardless of the SDLC model. Shifting left minimizes any technical debt that would require remediating early security flaws late in development or after the software is in production. Shifting left can also result in software with stronger security and resiliency.

There are many existing documents on secure software development practices, including those listed in the References section. This document does not introduce new practices or define new terminology. Instead, it describes a set of high-level practices based on established standards, guidance, and secure software development practice documents. These practices, collectively called the Secure Software Development Framework (SSDF), are intended to help the target audiences achieve secure software development objectives. Many of the practices directly involve the software itself, while others indirectly involve it (e.g., securing the development environment).

Future work may expand on this publication and potentially cover topics such as how the SSDF may apply to and vary for particular software development methodologies and associated practices like DevOps, how an organization can transition from their current software development practices to also incorporating the SSDF practices, and how the SSDF could be applied in the context of open-source software. Future work will likely take the form of use cases so that the insights will be more readily applicable to specific types of development environments, and it will likely include collaboration with the open-source community and other groups and organizations.

This document identifies secure software development practices but does not prescribe how to implement them. The focus is on the outcomes of the practices to be implemented rather than on

[1] Note that SDLC is also widely used for "system development life cycle." All usage of "SDLC" in this document is referencing software, not systems.

This publication is available free of charge from: https://doi.org/10.6028/NIST.SP.800-218

the tools, techniques, and mechanisms used to do so. Advantages of specifying the practices at a high level include the following:

- Can be used by organizations in any sector or community, regardless of size or cybersecurity sophistication
- Can be applied to software developed to support information technology (IT), industrial control systems (ICS), cyber-physical systems (CPS), or the Internet of Things (IoT)
- Can be integrated into any existing software development workflow and automated toolchain; should not negatively affect organizations that already have robust secure software development practices in place
- Makes the practices broadly applicable, not specific to particular technologies, platforms, programming languages, SDLC models, development environments, operating environments, tools, etc.
- Can help an organization document its secure software development practices today and define its future target practices as part of its continuous improvement process
- Can assist an organization currently using a classic software development model in transitioning its secure software development practices for use with a modern software development model (e.g., agile, DevOps)
- Can assist organizations that are procuring and using software to understand secure software development practices employed by their suppliers

This document provides a common language to describe fundamental secure software development practices. This is similar to the approach taken by the *Framework for Improving Critical Infrastructure Cybersecurity*, also known as the NIST Cybersecurity Framework [NISTCSF].[2] Expertise in secure software development is not required to understand the practices. The common language helps facilitate communications about secure software practices among both internal and external organizational stakeholders, such as:

- Business owners, software developers, project managers and leads, cybersecurity professionals, and operations and platform engineers within an organization who need to clearly communicate with each other about secure software development
- Software acquirers, including federal agencies and other organizations, that want to define required or desired characteristics for software in their acquisition processes in order to have higher-quality software (particularly with fewer significant security vulnerabilities)[3]

[2] The SSDF practices may help support the NIST Cybersecurity Framework Functions, Categories, and Subcategories, but the SSDF practices do not map to them and are typically the responsibility of different parties. Developers can adopt SSDF practices, and the outcomes of their work could help organizations with their operational security in support of the Cybersecurity Framework.

[3] Future work may provide more practical guidance for software acquirers on how they can leverage the SSDF in specific use cases.

This publication is available free of charge from: https://doi.org/10.6028/NIST.SP.800-218

- Software producers (e.g., commercial-off-the-shelf [COTS] product vendors, government-off-the-shelf [GOTS] software developers, software developers working within or on behalf of software acquirer organizations) that want to integrate secure software development practices throughout their SDLCs, express their secure software practices to their customers, or define requirements for their suppliers

This document's practices are not based on the assumption that all organizations have the same security objectives and priorities. Rather, the recommendations reflect that each software producer may have unique security assumptions, and each software acquirer may have unique security needs and requirements. While the aim is for each software producer to follow all applicable practices, the expectation is that the degree to which each practice is implemented and the formality of the implementation will vary based on the producer's security assumptions. The practices provide flexibility for implementers, but they are also clear to avoid leaving too much open to interpretation.

Although most of these practices are relevant to any software development effort, some are not. For example, if developing a particular piece of software does not involve using a compiler, there would be no need to follow a practice on configuring the compiler to improve executable security. Some practices are foundational, while others are more advanced and depend on certain foundational practices already being in place. Also, practices are not all equally important for all cases.

Factors such as risk, cost, feasibility, and applicability should be considered when deciding which practices to use and how much time and resources to devote to each practice.[4] Automatability is also an important factor to consider, especially for implementing practices at scale. The practices, tasks, and implementation examples represent a starting point to consider; they are meant to be changed and customized, and they are not prioritized. Any stated frequency for performing practices is notional. The intention of the SSDF is not to create a checklist to follow, but to provide a basis for planning and implementing a risk-based approach to adopting secure software development practices.

The responsibility for implementing the practices may be distributed among different organizations based on the delivery of the software and services (e.g., infrastructure as a service, software as a service, platform as a service, container as a service, serverless). In these situations, it likely follows a shared responsibility model involving the platform/service providers and the tenant organization that is consuming those platforms/services. The tenant organization should establish an agreement with the providers that specifies which party is responsible for each practice and task and how each provider will attest to their conformance with the agreement.

[4] Organizations seeking guidance on how to get started with secure software development can consult many publicly available references, such as "SDL That Won't Break the Bank" by Steve Lipner from SAFECode (https://i.blackhat.com/us-18/Thu-August-9/us-18-Lipner-SDL-For-The-Rest-Of-Us.pdf), "Application Software Security and the CIS Controls: A Reference Paper" by Steve Lipner and Stacy Simpson from SAFECode (https://safecode.org/resource-publications/cis-controls/), and "Simplified Implementation of the Microsoft SDL" by Microsoft (https://www.microsoft.com/en-us/download/details.aspx?id=12379).

This publication is available free of charge from: https://doi.org/10.6028/NIST.SP.800-218

2 The Secure Software Development Framework

This document defines version 1.1 of the Secure Software Development Framework (SSDF) with fundamental, sound, and secure recommended practices based on established secure software development practice documents. The practices are organized into four groups:

1. **Prepare the Organization (PO):** Organizations should ensure that their people, processes, and technology are prepared to perform secure software development at the organization level. Many organizations will find some PO practices to also be applicable to subsets of their software development, like individual development groups or projects.
2. **Protect the Software (PS):** Organizations should protect all components of their software from tampering and unauthorized access.
3. **Produce Well-Secured Software (PW):** Organizations should produce well-secured software with minimal security vulnerabilities in its releases.
4. **Respond to Vulnerabilities (RV):** Organizations should identify residual vulnerabilities in their software releases and respond appropriately to address those vulnerabilities and prevent similar ones from occurring in the future.

Each practice definition includes the following elements:

- **Practice:** The name of the practice and a unique identifier, followed by a brief explanation of what the practice is and why it is beneficial
- **Tasks:** One or more actions that may be needed to perform a practice
- **Notional Implementation Examples:** One or more notional examples of types of tools, processes, or other methods that could be used to help implement a task. No examples or combination of examples are required, and the stated examples are not the only feasible options. Some examples may not be applicable to certain organizations and situations.
- **References:** Pointers to one or more established secure development practice documents and their mappings to a particular task. Not all references will apply to all instances of software development.

Table 1 defines the practices. They are only a **subset** of what an organization may need to do. The information in the table is space constrained; much more information on each practice can be found in the references. Note that the order of the practices, tasks, and notional implementation examples in the table is not intended to imply the sequence of implementation or the relative importance of any practice, task, or example.

The table uses terms like “sensitive data,” “qualified person,” and “well-secured,” which are not defined in this publication. Organizations adopting the SSDF should define these terms in the context of their own environments and use cases. The same is true for the names of environments, like “development,” “build,” “staging,” “integration,” “test,” “production,” and “distribution,” which vary widely among organizations and projects. Enumerating your environments is necessary in order to secure them properly, and especially to prevent lateral movement of attackers from environment to environment.

This publication is available free of charge from: https://doi.org/10.6028/NIST.SP.800-218

Table 1: The Secure Software Development Framework (SSDF) Version 1.1

Practices	Tasks	Notional Implementation Examples	References
Prepare the Organization (PO)			
Define Security Requirements for Software Development (PO.1): Ensure that security requirements for software development are known at all times so that they can be taken into account throughout the SDLC and duplication of effort can be minimized because the requirements information can be collected once and shared. This includes requirements from internal sources (e.g., the organization's policies, business objectives, and risk management strategy) and external sources (e.g., applicable laws and regulations).	**PO.1.1**: Identify and document all security requirements for the organization's software development infrastructures and processes, and maintain the requirements over time.	**Example 1**: Define policies for securing software development infrastructures and their components, including development endpoints, throughout the SDLC and maintaining that security. **Example 2**: Define policies for securing software development processes throughout the SDLC and maintaining that security, including for open-source and other third-party software components utilized by software being developed. **Example 3**: Review and update security requirements at least annually, or sooner if there are new requirements from internal or external sources, or a major security incident targeting software development infrastructure has occurred. **Example 4**: Educate affected individuals on impending changes to requirements.	**BSAFSS**: SM.3, DE.1, IA.1, IA.2 **BSIMM**: CP1.1, CP1.3, SR1.1, SR2.2, SE1.2, SE2.6 **EO14028**: 4e(ix) **IEC62443**: SM-7, SM-9 **NISTCSF**: ID.GV-3 **OWASPASVS**: 1.1.1 **OWASPMASVS**: 1.10 **OWASPSAMM**: PC1-A, PC1-B, PC2-A **PCISSLC**: 2.1, 2.2 **SCFPSSD**: Planning the Implementation and Deployment of Secure Development Practices **SP80053**: SA-1, SA-8, SA-15, SR-3 **SP800160**: 3.1.2, 3.2.1, 3.2.2, 3.3.1, 3.4.2, 3.4.3 **SP800161**: SA-1, SA-8, SA-15, SR-3 **SP800181**: T0414; K0003, K0039, K0044, K0157, K0168, K0177, K0211, K0260, K0261, K0262, K0524; S0010, S0357, S0368; A0033, A0123, A0151
	PO.1.2: Identify and document all security requirements for organization-developed software to meet, and maintain the requirements over time.	**Example 1**: Define policies that specify risk-based software architecture and design requirements, such as making code modular to facilitate code reuse and updates; isolating security components from other components during execution; avoiding undocumented commands and settings; and providing features that will aid software acquirers with the secure deployment, operation, and maintenance of the software. **Example 2**: Define policies that specify the security requirements for the organization's software, and verify compliance at key points in the SDLC (e.g., classes of software flaws verified by gates, responses to vulnerabilities discovered in released software). **Example 3**: Analyze the risk of applicable technology stacks (e.g., languages, environments, deployment models), and recommend or require the use of stacks that will reduce risk compared to others. **Example 4**: Define policies that specify what needs to be archived for each software release (e.g., code, package files, third-party libraries, documentation, data inventory) and how long it needs to be retained based on the SDLC model, software end-of-life, and other factors. **Example 5**: Ensure that policies cover the entire software life cycle, including notifying users of the impending end of software support and the date of software end-of-life. **Example 6**: Review all security requirements at least annually, or sooner if there are new requirements from internal or external sources, a major vulnerability is discovered in released software, or a major security incident targeting organization-developed software has occurred. **Example 7**: Establish and follow processes for handling requirement exception requests, including periodic reviews of all approved exceptions.	**BSAFSS**: SC.1-1, SC.2, PD.1-1, PD.1-2, PD.1-3, PD.2-2, SI, PA, CS, AA, LO, EE **BSIMM**: SM1.1, SM1.4, SM2.2, CP1.1, CP1.2, CP1.3, CP2.1, CP2.3, AM1.2, SFD1.1, SFD2.1, SFD3.2, SR1.1, SR1.3, SR2.2, SR3.3, SR3.4 **EO14028**: 4e(ix) **IEC62443**: SR-3, SR-4, SR-5, SD-4 **ISO27034**: 7.3.2 **MSSDL**: 2, 5 **NISTCSF**: ID.GV-3 **OWASPMASVS**: 1.12 **OWASPSAMM**: PC1-A, PC1-B, PC2-A, PC3-A, SR1-A, SR1-B, SR2-B, SA1-B, IR1-A **PCISSLC**: 2.1, 2.2, 2.3, 3.3 **SCFPSSD**: Establish Coding Standards and Conventions **SP80053**: SA-8, SA-8(3), SA-15, SR-3 **SP800160**: 3.1.2, 3.2.1, 3.3.1 **SP800161**: SA-8, SA-15, SR-3 **SP800181**: T0414; K0003, K0039, K0044, K0157, K0168, K0177, K0211, K0260, K0261, K0262, K0524; S0010, S0357, S0368; A0033, A0123, A0151
	PO.1.3: Communicate requirements to all third parties who will provide commercial software components to the organization for reuse by the organization's own software. [Formerly PW.3.1]	**Example 1**: Define a core set of security requirements for software components, and include it in acquisition documents, software contracts, and other agreements with third parties. **Example 2**: Define security-related criteria for selecting software; the criteria can include the third party's vulnerability disclosure program and product security incident response capabilities or the third party's adherence to organization-defined practices. **Example 3**: Require third parties to attest that their software complies with the organization's security requirements.	**BSAFSS**: SM.1, SM.2, SM.2-1, SM.2-4 **BSIMM**: CP2.4, CP3.2, SR2.5, SR3.2 **EO14028**: 4e(vi), 4e(ix) **IDASOAR**: 19**,** 21 **IEC62443**: SM-9, SM-10 **MSSDL**: 7 **NISTCSF**: ID.SC-3 **OWASPSAMM**: SR3-A

Practices	Tasks	Notional Implementation Examples	References
		Example 4: Require third parties to provide provenance[5] data and integrity verification mechanisms for all components of their software. **Example 5**: Establish and follow processes to address risk when there are security requirements that third-party software components to be acquired do not meet; this should include periodic reviews of all approved exceptions to requirements.	**SCAGILE**: Tasks Requiring the Help of Security Experts 8 **SCFPSSD**: Manage Security Risk Inherent in the Use of Third-Party Components **SCSIC**: Vendor Sourcing Integrity Controls **SP80053**: SA-4, SA-9, SA-10, SA-10(1), SA-15, SR-3, SR-4, SR-5 **SP800160**: 3.1.1, 3.1.2 **SP800161**: SA-4, SA-9, SA-9(1), SA-9(3), SA-10, SA-10(1), SA-15, SR-3, SR-4, SR-5 **SP800181**: T0203, T0415; K0039; S0374; A0056, A0161
Implement Roles and Responsibilities (PO.2): Ensure that everyone inside and outside of the organization involved in the SDLC is prepared to perform their SDLC-related roles and responsibilities throughout the SDLC.	**PO.2.1**: Create new roles and alter responsibilities for existing roles as needed to encompass all parts of the SDLC. Periodically review and maintain the defined roles and responsibilities, updating them as needed.	**Example 1**: Define SDLC-related roles and responsibilities for all members of the software development team. **Example 2**: Integrate the security roles into the software development team. **Example 3**: Define roles and responsibilities for cybersecurity staff, security champions, project managers and leads, senior management, software developers, software testers, software assurance leads and staff, product owners, operations and platform engineers, and others involved in the SDLC. **Example 4**: Conduct an annual review of all roles and responsibilities. **Example 5**: Educate affected individuals on impending changes to roles and responsibilities, and confirm that the individuals understand the changes and agree to follow them. **Example 6**: Implement and use tools and processes to promote communication and engagement among individuals with SDLC-related roles and responsibilities, such as creating messaging channels for team discussions. **Example 7**: Designate a group of individuals or a team as the code owner for each project.	**BSAFSS**: PD.2-1, PD.2-2 **BSIMM**: SM1.1, SM2.3, SM2.7, CR1.7 **EO14028**: 4e(ix) **IEC62443**: SM-2, SM-13 **NISTCSF**: ID.AM-6, ID.GV-2 **PCISSLC**: 1.2 **SCSIC**: Vendor Software Development Integrity Controls **SP80053**: SA-3 **SP800160**: 3.2.1, 3.2.4, 3.3.1 **SP800161**: SA-3 **SP800181**: K0233
	PO.2.2: Provide role-based training for all personnel with responsibilities that contribute to secure development. Periodically review personnel proficiency and role-based training, and update the training as needed.	**Example 1**: Document the desired outcomes of training for each role. **Example 2**: Define the type of training or curriculum required to achieve the desired outcome for each role. **Example 3**: Create a training plan for each role. **Example 4**: Acquire or create training for each role; acquired training may need to be customized for the organization. **Example 5**: Measure outcome performance to identify areas where changes to training may be beneficial.	**BSAFSS**: PD.2-2 **BSIMM**: T1.1, T1.7, T1.8, T2.5, T2.8, T2.9, T3.1, T3.2, T3.4 **EO14028**: 4e(ix) **IEC62443**: SM-4 **MSSDL**: 1 **NISTCSF**: PR.AT **OWASPSAMM**: EG1-A, EG2-A **PCISSLC**: 1.3 **SCAGILE**: Operational Security Tasks 14, 15; Tasks Requiring the Help of Security Experts 1 **SCFPSSD**: Planning the Implementation and Deployment of Secure Development Practices **SCSIC**: Vendor Software Development Integrity Controls **SP80053**: SA-8 **SP800160**: 3.2.4, 3.2.6 **SP800161**: SA-8 **SP800181**: OV-TEA-001, OV-TEA-002; T0030, T0073, T0320; K0204, K0208, K0220, K0226, K0243, K0245, K0252; S0100, S0101; A0004, A0057

[5] *Provenance* is "the chronology of the origin, development, ownership, location, and changes to a system or system component and associated data. It may also include personnel and processes used to interact with or make modifications to the system, component, or associated data" [SP80053].

Practices	Tasks	Notional Implementation Examples	References
	PO.2.3: Obtain upper management or authorizing official commitment to secure development, and convey that commitment to all with development-related roles and responsibilities.	**Example 1**: Appoint a single leader or leadership team to be responsible for the entire secure software development process, including being accountable for releasing software to production and delegating responsibilities as appropriate. **Example 2**: Increase authorizing officials' awareness of the risks of developing software without integrating security throughout the development life cycle and the risk mitigation provided by secure development practices. **Example 3**: Assist upper management in incorporating secure development support into their communications with personnel with development-related roles and responsibilities. **Example 4**: Educate all personnel with development-related roles and responsibilities on upper management's commitment to secure development and the importance of secure development to the organization.	**BSIMM**: SM1.3, SM2.7, CP2.5 **EO14028**: 4e(ix) **NISTCSF**: ID.RM-1, ID.SC-1 **OWASPSAMM**: SM1.A **PCISSLC**: 1.1 **SP800181**: T0001, T0004
Implement Supporting Toolchains (PO.3): Use automation to reduce human effort and improve the accuracy, reproducibility, usability, and comprehensiveness of security practices throughout the SDLC, as well as provide a way to document and demonstrate the use of these practices. Toolchains and tools may be used at different levels of the organization, such as organization-wide or project-specific, and may address a particular part of the SDLC, like a build pipeline.	**PO.3.1**: Specify which tools or tool types must or should be included in each toolchain to mitigate identified risks, as well as how the toolchain components are to be integrated with each other.	**Example 1**: Define categories of toolchains, and specify the mandatory tools or tool types to be used for each category. **Example 2**: Identify security tools to integrate into the developer toolchain. **Example 3**: Define what information is to be passed between tools and what data formats are to be used. **Example 4**: Evaluate tools' signing capabilities to create immutable records/logs for auditability within the toolchain. **Example 5**: Use automated technology for toolchain management and orchestration.	**BSIMM**: CR1.4, ST1.4, ST2.5, SE2.7 **CNCFSSCP**: Securing Materials—Verification; Securing Build Pipelines—Verification, Automation, Secure Authentication/Access; Securing Artefacts—Verification; Securing Deployments—Verification **EO14028**: 4e(iii), 4e(ix) **MSSDL**: 8 **OWASPSAMM**: IR2-B, ST2-B **SCAGILE**: Tasks Requiring the Help of Security Experts 9 **SCSIC**: Vendor Software Delivery Integrity Controls **SP80053**: SA-15 **SP800161**: SA-15 **SP800181**: K0013, K0178
	PO.3.2: Follow recommended security practices to deploy, operate, and maintain tools and toolchains.	**Example 1**: Evaluate, select, and acquire tools, and assess the security of each tool. **Example 2**: Integrate tools with other tools and existing software development processes and workflows. **Example 3**: Use code-based configuration for toolchains (e.g., pipelines-as-code, toolchains-as-code). **Example 4**: Implement the technologies and processes needed for reproducible builds. **Example 5**: Update, upgrade, or replace tools as needed to address tool vulnerabilities or add new tool capabilities. **Example 6**: Continuously monitor tools and tool logs for potential operational and security issues, including policy violations and anomalous behavior. **Example 7**: Regularly verify the integrity and check the provenance of each tool to identify potential problems. **Example 8**: See PW.6 regarding compiler, interpreter, and build tools. **Example 9**: See PO.5 regarding implementing and maintaining secure environments.	**BSAFSS**: DE.2 **BSIMM**: SR1.1, SR1.3, SR3.4 **CNCFSSCP**: Securing Build Pipelines—Verification, Automation, Controlled Environments, Secure Authentication/Access; Securing Artefacts—Verification, Automation, Controlled Environments, Encryption; Securing Deployments—Verification, Automation **EO14028**: 4e(i)(F), 4e(ii), 4e(iii), 4e(v), 4e(vi), 4e(ix) **IEC62443**: SM-7 **IR8397**: 2.2 **OWASPASVS**: 1.14.3, 1.14.4, 14.1, 14.2 **OWASPMASVS**: 7.9 **OWASPSCVS**: 3, 5 **SCAGILE**: Tasks Requiring the Help of Security Experts 9 **SCFPSSD**: Use Current Compiler and Toolchain Versions and Secure Compiler Options **SCSIC**: Vendor Software Delivery Integrity Controls **SP80053**: SA-15 **SP800161**: SA-15 **SP800181**: K0013, K0178
	PO.3.3: Configure tools to generate artifacts[6] of their support of secure software development practices as defined by the organization.	**Example 1**: Use existing tooling (e.g., workflow tracking, issue tracking, value stream mapping) to create an audit trail of the secure development-related actions that are performed for continuous improvement purposes. **Example 2**: Determine how often the collected information should be audited, and implement the necessary processes. **Example 3**: Establish and enforce security and retention policies for artifact data. **Example 4**: Assign responsibility for creating any needed artifacts that tools	**BSAFSS**: PD.1-5 **BSIMM**: SM1.4, SM3.4, SR1.3 **CNCFSSCP**: Securing Build Pipelines—Verification, Automation, Controlled Environments; Securing Artefacts—Verification **EO14028**: 4e(i)(F), 4e(ii), 4e(v), 4e(ix) **IEC62443**: SM-12, SI-2 **MSSDL**: 8

[6] An *artifact* is "a piece of evidence" [adapted from IR7692]. *Evidence* is "grounds for belief or disbelief; data on which to base proof or to establish truth or falsehood" [SP800160]. Artifacts provide records of secure software development practices.